IQ

IQsure

Gamified Insurance Literacy Platform

Learn Insurance. Earn Rewards. Save Money.

PROJECT REPORT

Submitted By:	Ajay Lingampalli
Program:	The Hartford Training Program
Project Type:	Capstone Project
Date:	March 2026
Organization:	Hartford India

Table of Contents

1. **Abstract**
2. **Problem Statement**
3. **Project Objectives**
4. **Technology Stack**
   - 4.1 Backend Technologies
   - 4.2 Frontend Technologies
5. **System Architecture**
   - 5.1 Backend Architecture (Controller-Service-Repository)
   - 5.2 Frontend Architecture
   - 5.3 Request Flow Example
   - 5.4 System Architecture Diagram
6. **Project Structure**
   - 6.1 Backend Structure
   - 6.2 Frontend Structure
7. **Core Business Logic Engines**
   - 7.1 Gamification Engine
   - 7.2 Dynamic Premium Discount Engine
   - 7.3 Claims Processing Pipeline
8. **Role-Based Module Explanations**
   - 8.1 Customer Module
   - 8.2 Admin Module
   - 8.3 Underwriter Module
   - 8.4 Claims Officer Module
9. **End-to-End Workflows**
   - 9.1 Gamification to Policy Purchase Flow
   - 9.2 Claim Submission and Settlement Flow
10. **Database Design and Entity Descriptions**
11. **API Documentation**
   - 11.1 Authentication
   - 11.2 Users
   - 11.3 Quizzes and Attempts
   - 11.4 Policies and Premiums
   - 11.5 Claims
   - 11.6 Badges, Rewards and Discount Rules
   - 11.7 Admin Pipeline
12. **Frontend Component Architecture**
   - 12.1 Public Pages
   - 12.2 Customer Pages
   - 12.3 Admin Pages
   - 12.4 Shared and Service Layer
13. **Key Features and Technical Highlights**
   - 13.1 Key Features
   - 13.2 Development Challenges and Solutions
14. **Setup and Installation Guide**
   - 14.1 Backend Setup
   - 14.2 Frontend Setup
   - 14.3 Running Tests
15. **Future Enhancements**
16. **Conclusion**

1. Abstract

IQsure is a gamified insurance literacy platform developed as a capstone project during The Hartford training program. The system addresses a fundamental gap in the insurance industry: the widespread lack of consumer understanding of insurance products. By making insurance education interactive and rewarding, IQsure incentivizes users to engage with educational content, directly linking their learning achievements to tangible financial benefits in the form of premium discounts.

The platform operates on a novel premise where user behaviour drives financial outcomes. Customers take interactive quizzes on insurance topics, earning points and unlocking badges as they progress. These gamification elements feed directly into a dynamic discount engine that calculates personalized premium reductions when a user applies for an insurance policy. The more a user learns, the less they pay.

Built on a Spring Boot 4.x backend with H2 embedded database and an Angular 21 frontend styled with SCSS and Ionic-inspired components, IQsure is a production-ready, full-stack web application supporting four distinct user roles: Customer, Admin, Underwriter, and Claims Officer. The architecture follows a clean Controller-Service-Repository pattern with JWT-based stateless authentication, comprehensive DTO separation, and Swagger-documented REST APIs.

2. Problem Statement

The insurance industry faces a persistent challenge: the majority of consumers are underinsured, hold inappropriate policies, or avoid insurance altogether due to poor understanding of what they are purchasing. Several interconnected problems drive this:

- **Insurance Illiteracy:** Most consumers lack a foundational understanding of insurance terminology, coverage types, exclusions, and policy structures, making informed purchasing nearly impossible.
- **Disengaged Education:** Traditional insurance education — brochures, static FAQs, and dense policy documents — fails to engage modern consumers, particularly younger demographics.
- **No Incentive to Learn:** There is no mechanism in the conventional insurance sales process that rewards customers for educating themselves, creating a negative cycle where ignorance persists.
- **Complex Premium Pricing:** Premium calculations are opaque and static, with customers having no agency over how their engagement or knowledge influences the price they pay.
- **Inefficient Claim Processing:** Without digital pipeline management, claim assignment and underwriting review become manual, error-prone processes prone to bottlenecks.
- **Lack of Gamification in Finance:** The financial services sector has largely failed to leverage points, badges, leaderboards, and rewards to drive positive consumer behaviour.

3. Project Objectives

The primary objectives of IQsure are to design and deliver a comprehensive, full-stack platform through the following targeted goals:

1. **Gamify Insurance Education:** Create an engaging quiz-based learning system covering Health, Life, and Motor insurance with dynamic scoring, speed bonuses, and a progressive badge and level system that rewards sustained engagement.
2. **Implement Dynamic Premium Discounts:** Build a configurable discount rule engine that automatically calculates personalized premium reductions based on quiz performance, accumulated points, and earned badges.
3. **Provide a Complete Insurance Policy Lifecycle:** Enable customers to browse policies, preview personalized premiums, purchase policies, add insured family members, and manage their active portfolio in a single unified platform.
4. **Automate Claims Processing Pipeline:** Implement a structured, role-aware pipeline for claim submission, assignment, review, approval or rejection, and settlement tracking.
5. **Enforce Role-Based Access and Security:** Secure all platform functionality with JWT authentication and granular role-based access control across four distinct roles.

6. Enable Full Administrative Control: Provide administrators with complete CRUD control over quizzes, questions, badges, rewards, discount rules, policies, and educational content, including dynamic configuration of gamification parameters.

4. Technology Stack

IQsure is designed as a decoupled full-stack web application with a Spring Boot REST API backend and an Angular SPA frontend. The two communicate exclusively via HTTP/JSON with JWT bearer tokens for authorization.

4.1 Backend Technologies

Layer / Category	Technology	Purpose and Details
Framework	Spring Boot 4.x	High-performance Java RESTful API with auto-configuration, embedded Tomcat, and production-ready features
Language	Java 17+	Modern Java with records, sealed classes, and enhanced switch expressions
Persistence	Spring Data JPA (Hibernate)	Repository pattern with auto-generated queries, entity relationship management, and schema migration
Database	H2 Embedded Database	File-based relational database for portability; accessible via H2 Console at /h2-console
Security	Spring Security + JJWT 0.13	Stateless JWT authentication, BCrypt password hashing, CORS configuration, role-based endpoint protection
API Documentation	SpringDoc OpenAPI (Swagger)	Auto-generated interactive API documentation at /swagger-ui.html
Code Generation	Lombok	Eliminates boilerplate: @Data, @Builder, @NoArgsConstructor, @AllArgsConstructor on all entities
Build Tool	Maven	Dependency management, build lifecycle, and production JAR packaging
Dev Tools	Spring Boot DevTools	Hot reload for rapid development iteration
Testing	Spring Boot Test + JUnit	Unit and integration testing via mvn test

4.2 Frontend Technologies

Layer / Category	Technology	Purpose and Details
Framework	Angular 21 (TypeScript 5.9)	Standalone component architecture, reactive forms, and Angular Router for SPA navigation
UI Styling	SCSS (Global + Component)	Custom styling with The Hartford brand palette — Claret Red (#75013F) and Fuchsia Pink (#FE3082)
Icons	Ionicons	Mobile-friendly icon set integrated for consistent visual language across the interface
State Management	RxJS 7.8 Observables	Reactive HTTP communication, async stream handling, and component data binding via Angular Services
Auth Flow	JWT + Auth Interceptor	AuthInterceptor automatically attaches Bearer tokens to all outbound HTTP requests
Route Protection	Angular Route Guards	AuthGuard, AdminGuard, UnderwriterGuard, and ClaimsOfficerGuard enforce role-based route access
Type Safety	TypeScript Interfaces (models.ts)	Centralized data model definitions ensuring type-safe API communication across all components
Testing	Vitest 4.x	Fast unit test runner for frontend components and services
Package Manager	npm 10.9	Dependency management and build scripts

5. System Architecture

IQsure follows a classic three-tier web architecture with a strict separation between the presentation layer (Angular SPA), the business logic layer (Spring Boot), and the data layer (H2 JPA). The two application layers communicate over HTTP REST using JSON payloads, secured by JWT tokens validated on every protected request.

5.1 Backend Architecture (Controller-Service-Repository)

- **Controller Layer:** Handles HTTP request mapping, input validation via @Valid annotations, and JSON serialization/deserialization. Each controller maps to a specific resource domain (AuthController, QuizController, PolicyController, ClaimController, etc.) and delegates business logic entirely to the service layer.
- **Service Layer:** Contains all business logic including score calculation, badge award automation, premium discount computation, claim status transitions, and notification dispatch. Services interact with repositories for data access and with each other for cross-domain logic.
- **Repository Layer:** Extends Spring Data JPA's JpaRepository, providing auto-generated CRUD operations and custom JPQL queries for complex data retrieval without boilerplate SQL.
- **Security Configuration:** SecurityConfig.java defines the filter chain, permitting public endpoints (/api/auth/**) and protecting all others. JWT tokens are validated via a custom filter on every request.
- **Data Seeding:** DataSeeder.java pre-populates the in-memory database on startup with sample quizzes, questions, badges, policies, and discount rules for immediate demonstration.

5.2 Frontend Architecture

- **Standalone Components:** All Angular components use the standalone architecture (no NgModules), imported directly in the router configuration for reduced boilerplate and faster lazy loading.
- **Service Layer:** ApiService is the single abstraction for all backend HTTP communication, exposing typed Observable-returning methods for every API endpoint. AuthService manages the JWT lifecycle, user identity, and role extraction.
- **Auth Interceptor:** auth.interceptor.ts intercepts all outgoing HttpClient requests and injects the Authorization: Bearer token header from localStorage, ensuring authenticated requests without per-call boilerplate.
- **Route Guards:** Four guards (AuthGuard, AdminGuard, UnderwriterGuard, ClaimsOfficerGuard) protect role-specific routes, redirecting unauthorized users to the login page.

5.3 Request Flow Example (User Takes a Quiz)

```
TakeQuizComponent → POST /api/v1/attempts + JWT → AuthFilter (validate token) →
AttemptController → QuizAttemptService (calculateScore, awardBadges, updatePoints) → JPA
Repositories → H2 Database → AttemptResponseDTO → QuizResultComponent
```

5.4 System Architecture Diagram

The diagram below presents the complete four-layer architecture of IQsure: Presentation (Angular 21 SPA with role-based dashboards), Application (REST API controllers, business logic services, and Spring Security), Infrastructure (JPA domain entities, DTO layer, and repository layer), and the Data layer (H2 embedded database, file storage, and DataSeeder). Arrows indicate the direction of dependency and data flow between layers.

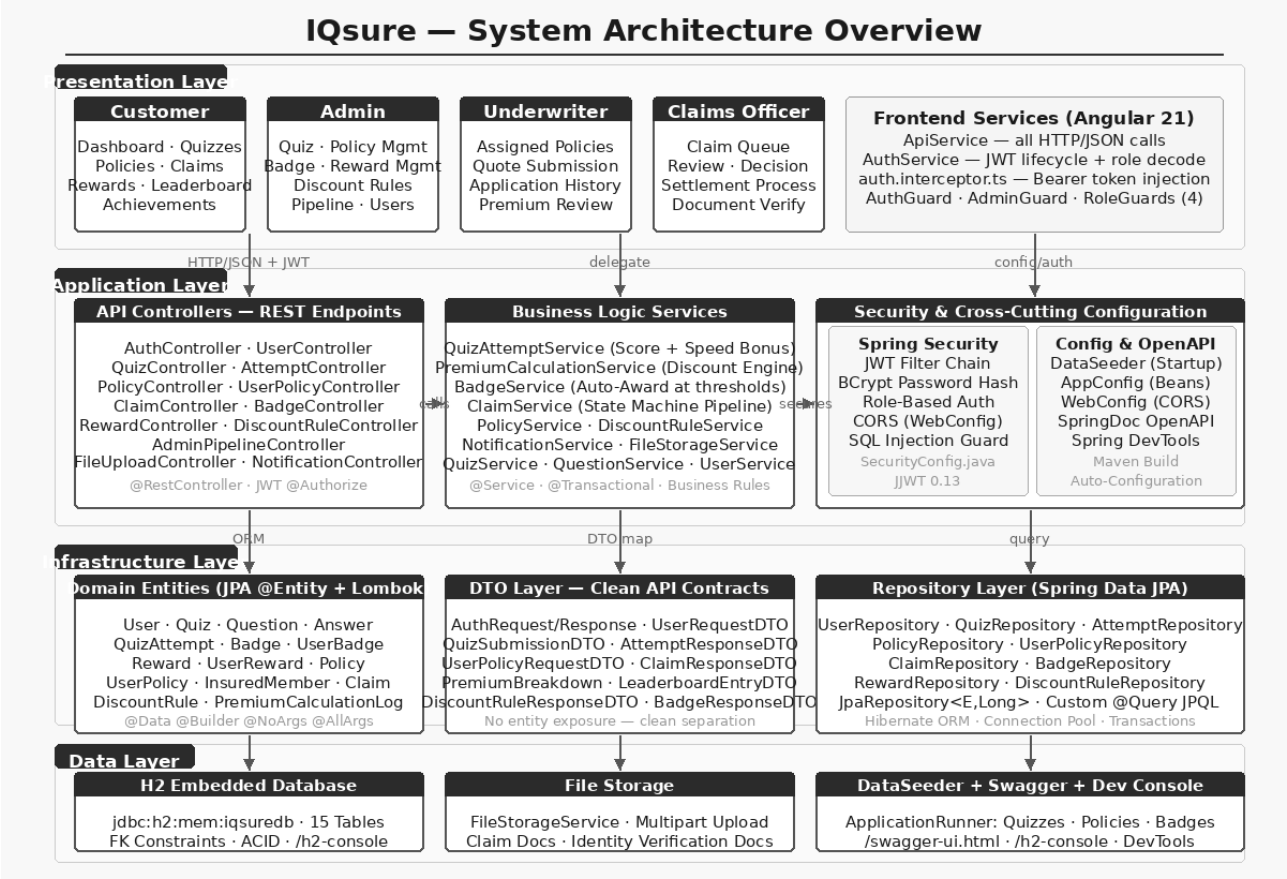

Figure 5.1 — IQsure Four-Layer System Architecture

6. Project Structure

The repository (HTF_IQsure_Capstone_Project) is organized into two top-level folders — a Maven-managed Spring Boot backend at the root level and an Angular frontend in the frontend/ directory — along with comprehensive documentation files.

6.1 Backend Structure

Package / Folder	Contents
org.hartford.iqsure.config	AppConfig.java (beans), SecurityConfig.java (JWT filter chain), WebConfig.java (CORS), DataSeeder.java (database population on startup)
org.hartford.iqsure.controller	15 REST controllers: AuthController, UserController, QuizController, AttemptController, QuestionController, PolicyController, UserPolicyController, ClaimController, BadgeController, RewardController, DiscountRuleController, AdminPipelineController, EducationContentController, NotificationController, FileUploadController
org.hartford.iqsure.entity	15 JPA entities: User, Quiz, Question, Answer, QuizAttempt, Badge, UserBadge, Reward, UserReward, Policy, UserPolicy, InsuredMember, Claim, DiscountRule, PremiumCalculationLog
org.hartford.iqsure.service	Business logic services: QuizAttemptService, PremiumCalculationService, ClaimService, PolicyService, BadgeService, DiscountRuleService, NotificationService, FileStorageService, and others
org.hartford.iqsure.dto	Request DTOs (QuizSubmissionDTO, UserPolicyRequestDTO, etc.) and Response DTOs (AttemptResponseDTO, ClaimResponseDTO, LeaderboardEntryDTO, etc.) for clean API contracts
org.hartford.iqsure.repository	JpaRepository interfaces with custom JPQL query methods for each entity
src/test/java	JUnit + Spring Boot Test integration tests via mvn test

6.2 Frontend Structure

Folder	Contents
src/app/pages/	Role-grouped page components: public (landing, about, login, register), customer (dashboard, quizzes, take-quiz, policies, my-policies, my-claims, leaderboard, rewards, achievements), admin (dashboard, quiz-mgmt, policy-mgmt, badge-mgmt, discount-rules, users, assign-underwriter, assign-claims-officer), underwriter, claims-officer
src/app/components/	Shared reusable components: NavbarComponent (role-adaptive navigation)
src/app/services/	ApiService (all HTTP calls), AuthService (JWT + role management), auth.interceptor.ts (token injection)
src/app/models/models.ts	Centralized TypeScript interface definitions for all API data models
src/app/guards/	AuthGuard, AdminGuard, UnderwriterGuard, ClaimsOfficerGuard for route protection
src/app/app.routes.ts	Centralized route configuration with lazy-loaded page components and guard associations

7. Core Business Logic Engines

7.1 Gamification Engine

The gamification engine is the central innovation of IQsure. It comprises four interconnected subsystems that together create a continuous engagement loop.

7.1.1 Quiz Scoring System

When a user submits a quiz via POST /api/v1/attempts, the QuizAttemptService processes the submission through a multi-step scoring pipeline:

- **Correctness Scoring:** 10 base points are awarded for each correct answer, compared against Answer entities in the database.
- **Speed Bonus:** The submission time is compared to the quiz time limit (5 minutes). Fast completions earn up to 50% additional points as a speed multiplier, incentivizing knowledge fluency over deliberate lookup.
- **First-Attempt Rule:** Points are only awarded on the first attempt at each quiz, preventing gaming the system through repeated attempts.
- **Streak Tracking:** Daily engagement is tracked as a streak counter, rewarding consistent users with bonus points and maintaining platform retention.

7.1.2 Badge Award Engine

The BadgeService evaluates badge eligibility automatically after every successful quiz attempt. Badges are awarded threshold-based on accumulated userPoints:

Badge	Points Required	Description
Quiz Beginner	10 points	Awarded on first correct quiz answer, welcoming new learners to the platform
Insurance Learner	50 points	Recognizes consistent early engagement across multiple quiz topics
Knowledge Pro	150 points	Marks advanced understanding, awarded to dedicated platform learners
IQsure Champion	300 points	The highest tier badge, signifying mastery of insurance literacy

7.1.3 Level and Achievement System

The level system provides continuous progression incentive. Users advance one level for every 100 points earned, with no upper ceiling. Eight unique achievements trigger on specific milestones (e.g., completing a first quiz, achieving a perfect score, reaching level 5) and award bonus points upon unlock, creating a layered progression system.

7.2 Dynamic Premium Discount Engine

The PremiumCalculationService is the core financial engine of IQsure. When a customer requests a premium calculation for a policy, the service executes the following multi-step discount evaluation:

Step	Logic
1. Fetch User Gamification Profile	Retrieve the user's current userPoints, count of earned badges, and best quiz score percentage from the database.
2. Fetch Applicable Discount Rules	Query all active DiscountRule records that apply to the requested policy type (PolicyType enum).
3. Evaluate Each Rule	For each DiscountRule, check whether the user meets all three thresholds: minUserPoints, minBadgesEarned, and minQuizScorePercent.
4. Stack Applicable Discounts	Accumulate discountPercentage values from all qualifying rules. The total discount is capped at 50% of the base premium to maintain business viability.

Step	Logic
5. Calculate Final Premium	FinalPremium = BasePremium × (1 – TotalDiscountPercent / 100). A PremiumCalculationLog is persisted recording the full calculation context.
6. Return PremiumBreakdown	A detailed breakdown DTO is returned to the frontend showing base premium, each applied rule and its discount, total savings, and final price.

7.3 Claims Processing Pipeline

The ClaimService manages the full lifecycle of an insurance claim through a series of enforced state transitions:

Status	Triggered By	Description
SUBMITTED	Customer via POST /api/v1/claims/file	Customer files claim with incident details, claim type (HEALTH/LIFE/MOTOR/TRAVEL), amount, and optionally uploaded documents.
ASSIGNED	Admin via PUT /pipeline/claims/{id}/assign	Admin assigns the claim to a specific Claims Officer user via the AdminPipelineController.
UNDER_REVIEW	Claims Officer starts review	The assigned officer opens the claim for detailed investigation and document verification.
APPROVED	Claims Officer via PUT /claims/{id}/process	Officer approves the claim with a reviewed approved amount recorded against the claim record.
REJECTED	Claims Officer via PUT /claims/{id}/process	Officer rejects the claim with a mandatory rejectionReason field for auditability.
SETTLED	Claims Officer via PUT /claims/{id}/settle	Officer confirms final settlement amount, closing the claim lifecycle.

8. Role-Based Module Explanations

IQsure implements four distinct user roles, each with a dedicated dashboard and exclusive functionality enforced by both frontend guards and backend Spring Security endpoint protection.

8.1 Customer Module

The Customer (ROLE_USER) is the primary user of IQsure and the beneficiary of its core value proposition.

- **Dashboard:** Personalized hub displaying current points, level progress bar, active streak, total badges earned, and quick-action shortcuts.
- **Quizzes:** Browse available quizzes filtered by category (Health, Life, Motor) and difficulty (Easy, Medium, Hard). Each quiz card shows estimated points and best previous score.
- **Take Quiz:** Timed interactive quiz interface with a 5-minute countdown, real-time question navigation, and immediate score reveal with points earned and badges unlocked.
- **Achievements:** Gallery of all 8 platform achievements showing earned status, unlock conditions, and bonus points awarded.
- **Leaderboard:** Ranked table of all users by total points. Top 3 users receive a podium display. Users see their own rank highlighted.
- **Policies:** Marketplace of all active insurance plans. Each policy card shows coverage type, base premium, age range, and plan type (Individual / Family / Senior Citizen).
- **Premium Preview:** Before purchasing, customers can preview their personalized premium including a transparent breakdown of every applied discount rule and total savings amount.
- **My Policies:** View all purchased policies with status (PENDING_UNDERWRITING / ACTIVE / EXPIRED / CANCELLED), payment details, and insured member list.
- **Claims:** Multi-step claim filing wizard and real-time status tracking of all submitted claims.
- **Rewards:** Catalog of redeemable rewards. Users spend accumulated points to redeem premium discount vouchers and cashback rewards with expiry dates.

8.2 Admin Module

The Admin (ROLE_ADMIN) has full platform management authority and exclusive access to all CRUD operations.

- **Dashboard:** System-wide overview of total users, active policies, pending claims, and quiz participation statistics.
- **Quiz Management:** Create, edit, and delete quizzes with full question and answer CRUD. Set difficulty, category, and point values.
- **Policy Management:** Design and update insurance product offerings, setting base premiums, coverage amounts, duration, age ranges, and coverage features.
- **Badge Management:** Create and configure achievement badges with point thresholds and descriptive metadata.
- **Discount Rules:** Define and activate discount rule logic, specifying minimum point/badge/score thresholds and corresponding discount percentages per policy type.
- **User Management:** View all users across all roles. Manage account statuses. Admin accounts are protected from deletion.
- **Pipeline:** Assign pending policy applications to Underwriters and view all claim submissions for assignment to Claims Officers.
- **Education Content:** Manage the learning center articles and videos providing supplementary insurance education alongside quizzes.

8.3 Underwriter Module

The Underwriter (ROLE_UNDERWRITER) reviews policy applications assigned by the Admin and determines final premium quotes.

- **Assigned Applications:** List of all UserPolicy records in PENDING_UNDERWRITING status assigned to this underwriter.
- **Quote Submission:** Review the customer's profile, requested policy, and pre-calculated discount premium. The underwriter can adjust the final premium quote based on their professional assessment before confirming activation.
- **Application History:** Full history of processed applications with outcomes for audit purposes.

8.4 Claims Officer Module

The Claims Officer (ROLE_CLAIMS_OFFICER) handles the investigation and resolution of insurance claims assigned by the Admin.

- **Claim Queue:** List of all claims assigned for review with claim type, amount, submission date, and current status.
- **Claim Detail:** Full view of incident description, claim type, requested amount, supporting documentation, policy coverage details, and insured member information.
- **Decision Interface:** Approve a claim with a reviewed settlement amount, or reject it with a mandatory reason. Status transitions are enforced sequentially.
- **Settlement Confirmation:** Confirm and record the final settlement amount, closing the claim.

9. End-to-End Workflows

9.1 Gamification to Policy Purchase Flow

1. Registration: Customer registers an account. User record is created with 0 points, ROLE_USER role, and no badges.

2. Quiz Participation: Customer selects a quiz, answers all questions within the 5-minute timer. QuizAttemptService calculates score including speed bonus. userPoints is updated on the User entity.

3. Badge Unlock: BadgeService checks all badge thresholds against new userPoints total. Qualifying badges are automatically awarded and recorded as UserBadge records.

4. Level Progression: Frontend calculates current level as floor(userPoints / 100). Dashboard progress bar updates immediately.

5. Achievement Triggers: Achievement milestones are checked and bonus points added for qualifying events (first quiz, perfect score, leaderboard entry, etc.).

6. Policy Discovery: Customer browses the policy marketplace and clicks Premium Preview on a policy of interest.

7. Discount Calculation: PremiumCalculationService evaluates all active DiscountRules against the customer's current gamification profile. A full breakdown is returned and displayed.

8. Policy Purchase: Customer confirms purchase. A UserPolicy record is created with PENDING_UNDERWRITING status. PremiumCalculationLog is persisted for audit.

9. Underwriter Assignment: Admin assigns the application to an Underwriter via the pipeline interface.

10. Quote and Activation: Underwriter reviews and sends a final quote. Customer accepts, makes payment, and policy status transitions to ACTIVE.

9.2 Claim Submission and Settlement Flow

1. Claim Filing: Customer files a claim against an ACTIVE UserPolicy, specifying ClaimType (HEALTH, LIFE, MOTOR, TRAVEL), incident date, description, and requested claim amount.

2. Initial Validation: ClaimService validates that the referenced UserPolicy is in ACTIVE status. A unique claimNumber is generated and the Claim record is persisted with SUBMITTED status.

3. Admin Assignment: Admin reviews the claim queue via the pipeline and assigns it to the least-loaded available Claims Officer.

4. Officer Review: Claims Officer reviews all claim details, verifies supporting documents via the FileUploadController-served files, and assesses claim validity.

5. Decision: Officer approves with an approvedAmount or rejects with a rejectionReason. Claim status updates accordingly and the customer receives a Notification.

6. Settlement: For approved claims, the officer confirms final settlementAmount, transitioning the claim to SETTLED status and completing the lifecycle.

10. Database Design and Entity Descriptions

The IQsure database uses H2 embedded relational storage managed by Spring Data JPA (Hibernate). All entities use auto-incremented Long primary keys. The schema is organized into four logical domains: Identity, LMS (Learning Management), Gamification, and Insurance.

10.1 Entity: User

Central identity and profile entity for all system participants. Role enum differentiates: ROLE_USER, ROLE_ADMIN, ROLE_UNDERWRITER, ROLE_CLAIMS_OFFICER.

Field	Type / Constraint	Description
userId	Long (PK, auto-increment)	Unique user identifier.
name	String, NOT NULL	Full display name of the user.
email	String, UNIQUE, NOT NULL	Login identifier. Must be unique platform-wide.
password	String, NOT NULL	BCrypt-hashed password. Plain-text never persisted.
phone	String, nullable	Optional contact phone number.
userPoints	Integer, default 0	Accumulated gamification points from all quiz attempts.
role	Enum (Role), NOT NULL	Access role: ROLE_USER, ROLE_ADMIN, ROLE_UNDERWRITER, ROLE_CLAIMS_OFFICER.
licenseNumber	String, nullable	Professional license number for Underwriter and Claims Officer users.
specialization	String, nullable	Area of specialization for professional role users.
commissionPercentage	BigDecimal, nullable	Commission rate applicable to Underwriter users.

Relationships: One User has many QuizAttempts, UserBadge records, UserReward records, and UserPolicy records. One User (as Underwriter) is assigned to many UserPolicies. One User (as Claims Officer) is assigned to many Claims.

10.2 Entity: Quiz

Defines an insurance education quiz within the Learning Management domain.

Field	Type / Constraint	Description
quizId	Long (PK)	Unique quiz identifier.
title	String, NOT NULL	Descriptive quiz title, e.g. Life Insurance Fundamentals.
category	String, NOT NULL	Topic category: Health, Life, or Motor.
difficulty	Enum (Difficulty)	Difficulty level: EASY, MEDIUM, or HARD.
questions	List	One-to-many: all questions belonging to this quiz.
attempts	List	One-to-many: all user attempts recorded for this quiz.

10.3 Entity: Question

Represents a single question within a quiz.

Field	Type / Constraint	Description
questionId	Long (PK)	Unique question identifier.
quiz	ManyToOne → Quiz	Parent quiz this question belongs to.
text	String, NOT NULL	The question text presented to the user.
options	String	Serialized multiple-choice option labels.
answers	List	One-to-many: answer choices for this question.

Field	Type / Constraint	Description
explanation	String	Post-answer explanation shown to the user after submitting.

10.4 Entity: Answer

Represents one answer choice for a Question, including the correctness marker.

Field	Type / Constraint	Description
answerId	Long (PK)	Unique answer identifier.
question	ManyToOne → Question	Parent question this answer belongs to.
text	String, NOT NULL	The answer option text displayed to the user.
rightOption	Integer	Index of the correct answer option (0-based).

10.5 Entity: QuizAttempt

Records a single user's attempt at a quiz, including score and points earned.

Field	Type / Constraint	Description
attemptId	Long (PK)	Unique attempt identifier.
user	ManyToOne → User	The user who made this attempt.
quiz	ManyToOne → Quiz	The quiz that was attempted.
score	Integer	Number of correct answers submitted.
totalQuestions	Integer	Total number of questions in the quiz at attempt time.
pointsEarned	Integer, default 0	Points awarded for this attempt including speed bonus (first attempt only).
attemptDate	LocalDateTime	Timestamp of when the attempt was submitted.

10.6 Entity: Badge

Defines an achievement badge that users can earn by reaching point thresholds.

Field	Type / Constraint	Description
badgeId	Long (PK)	Unique badge identifier.
name	String, NOT NULL	Badge display name, e.g. Quiz Beginner or IQsure Champion.
description	String	Human-readable description of what this badge signifies.
reqPoints	Integer, NOT NULL	Minimum userPoints required to earn this badge.
icon	String	Icon identifier or URL for the badge visual.

10.7 Entity: UserBadge

Junction entity recording which badges a specific user has earned and when.

Field	Type / Constraint	Description
id	Long (PK)	Unique junction record identifier.
user	ManyToOne → User	The user who earned the badge.
badge	ManyToOne → Badge	The badge that was earned.
earnedDate	LocalDateTime, default now()	Timestamp of when the badge was automatically awarded.

10.8 Entity: Reward

Defines a redeemable reward that users can claim by spending their accumulated points.

Field	Type / Constraint	Description
rewardId	Long (PK)	Unique reward identifier.
rewardType	String, NOT NULL	Category of reward: PREMIUM_DISCOUNT or CASHBACK.
discountValue	Double, NOT NULL	Financial value of the reward (percentage or currency amount).
description	String	Human-readable description of the reward benefit.
reqPoints	Integer, NOT NULL	Points cost to redeem this reward.
expiryDate	LocalDate	Date after which the reward can no longer be redeemed.

10.9 Entity: UserReward

Junction entity tracking reward redemptions by users.

Field	Type / Constraint	Description
id	Long (PK)	Unique redemption record identifier.
user	ManyToOne → User	The user who redeemed the reward.
reward	ManyToOne → Reward	The reward that was redeemed.
redeemedDate	LocalDateTime, default now()	Timestamp of redemption.
used	boolean, default false	Whether the redeemed reward has been applied to a policy purchase.

10.10 Entity: Policy

Defines an insurance product available for purchase on the platform.

Field	Type / Constraint	Description
policyId	Long (PK)	Unique policy identifier.
title	String, NOT NULL	Product display name, e.g. Gold Health Plan.
description	String	Detailed coverage description shown to customers.
policyType	Enum (PolicyType)	Coverage type: HEALTH, LIFE, MOTOR, or TRAVEL.
basePremium	Double, NOT NULL	Base annual premium before any discount application.
coverageAmount	Double, NOT NULL	Maximum insurance payout available under this policy.
durationMonths	Integer, NOT NULL	Standard policy duration in months.
isActive	Boolean, default true	Whether the policy is currently available for purchase.
ageRange	String	Eligible customer age range, e.g. 18-65.
planType	String	Plan category: INDIVIDUAL, FAMILY, or SENIOR_CITIZEN.
waitingPeriod	String	Initial waiting period before coverage activates, e.g. 1 month.
hasMaternityCover	Boolean	Whether the policy includes maternity coverage.
hasPreExistingCover	Boolean	Whether the policy covers pre-existing medical conditions.

10.11 Entity: DiscountRule

Defines an admin-configurable discount rule evaluated by the PremiumCalculationService.

Field	Type / Constraint	Description
ruleId	Long (PK)	Unique rule identifier.
ruleName	String, UNIQUE, NOT NULL	Unique name identifying the rule, e.g. Quiz Master Discount.

Field	Type / Constraint	Description
description	String	Human-readable explanation of the rule conditions and benefit.
minQuizScorePercent	Double, default 0.0	Minimum best-quiz-score percentage the user must have achieved.
minUserPoints	Integer, default 0	Minimum accumulated userPoints the user must hold.
minBadgesEarned	Integer, default 0	Minimum number of badges the user must have earned.
discountPercentage	Double, NOT NULL	Percentage discount to apply to the base premium if all thresholds are met.
applicablePolicyType	Enum (PolicyType)	The policy type this rule applies to (HEALTH, LIFE, MOTOR, or TRAVEL).
isActive	Boolean, default true	Whether this rule is currently being evaluated in premium calculations.

10.12 Entity: UserPolicy

Intersection entity representing a policy purchased by a specific user. Tracks the full lifecycle from application through underwriting to active coverage.

Field	Type / Constraint	Description
id	Long (PK)	Unique user-policy record identifier.
user	ManyToOne → User	The customer who purchased this policy.
policy	ManyToOne → Policy	The insurance product purchased.
underwriter	ManyToOne → User, nullable	The Underwriter assigned to review this application.
finalPremium	Double, NOT NULL	Actual premium paid after all discounts applied.
discountApplied	Double, NOT NULL	Total discount percentage applied to the base premium.
purchaseDate	LocalDateTime, default now()	Timestamp of initial policy application.
status	Enum (PolicyStatus)	Lifecycle state: PENDING_UNDERWRITING, ACTIVE, EXPIRED, or CANCELLED.
insuredMembers	List	Family members covered under this policy (for family plans).
claims	List	All claims filed against this policy.

10.13 Entity: InsuredMember

Records additional family members covered under a UserPolicy, relevant for FAMILY and SENIOR_CITIZEN plan types.

Field	Type / Constraint	Description
id	Long (PK)	Unique member record identifier.
userPolicy	ManyToOne → UserPolicy	The policy under which this member is covered.
fullName	String, NOT NULL	Full legal name of the insured member.
relationship	String, NOT NULL	Relationship to the policyholder, e.g. Spouse, Child, Parent.
dateOfBirth	LocalDate	Date of birth for age-based coverage validation.
gender	String	Gender of the insured member.
preExistingConditions	String	Description of any pre-existing medical conditions disclosed.

10.14 Entity: Claim

Records an insurance claim filed by a customer against an active UserPolicy, tracking its full lifecycle from submission to settlement.

Field	Type / Constraint	Description
id	Long (PK)	Unique claim identifier.
claimNumber	String, UNIQUE	Auto-generated human-readable claim reference number.
userPolicy	ManyToOne → UserPolicy	The policy against which the claim is filed.
user	ManyToOne → User	The customer who filed the claim.
type	Enum (ClaimType)	Type of claim: HEALTH, LIFE, MOTOR, or TRAVEL.
amount	BigDecimal, NOT NULL	Amount of compensation requested by the customer.
approvedAmount	BigDecimal, nullable	Amount approved by the Claims Officer after review.
settlementAmount	BigDecimal, nullable	Final settlement amount confirmed on claim closure.
hospitalName	String, nullable	Hospital name for HEALTH claims.
incidentDate	LocalDate, NOT NULL	Date the insured incident occurred.

Field	Type / Constraint	Description
diagnosis	String, nullable	Medical diagnosis for HEALTH claims.
status	Enum (ClaimStatus)	Lifecycle state: SUBMITTED, ASSIGNED, UNDER_REVIEW, APPROVED, REJECTED, SETTLED.
rejectionReason	String, nullable	Mandatory reason provided when a claim is rejected.
assignedOfficer	ManyToOne → User, nullable	Claims Officer assigned to review this claim.
reviewStartedAt	LocalDateTime, nullable	Timestamp of when the officer began review.

10.15 Entity: PremiumCalculationLog

Immutable audit log recording every premium calculation event for regulatory traceability and customer transparency.

Field	Type / Constraint	Description
logId	Long (PK)	Unique log entry identifier.
user	ManyToOne → User	The user for whom the premium was calculated.
policy	ManyToOne → Policy	The policy for which the premium was calculated.
basePremium	Double	Base premium before any discounts.
totalDiscountPercent	Double	Total combined discount percentage applied.
finalPremium	Double	Final premium after all discounts.
userPointsSnapshot	Integer	User's point total at the time of calculation (for audit).
badgeCountSnapshot	Integer	User's badge count at the time of calculation.
bestQuizScoreSnapshot	Double	User's best quiz score percentage at time of calculation.
appliedRuleNames	String	Comma-separated names of all discount rules that were applied.
calculatedAt	LocalDateTime, default now()	Timestamp of the calculation event.

10.16 Entity Relationship Summary

Entity	Relationship	Related Entity	Cardinality
User	attempts	QuizAttempt	1 to Many
Quiz	contains	Question	1 to Many
Question	has	Answer	1 to Many
User	earns	UserBadge (→ Badge)	1 to Many (junction)
User	redeems	UserReward (→ Reward)	1 to Many (junction)
User	purchases	UserPolicy (→ Policy)	1 to Many (junction)
User (Underwriter)	underwrites	UserPolicy	1 to Many
UserPolicy	includes	InsuredMember	1 to Many
UserPolicy	raises	Claim	1 to Many
User (ClaimsOfficer)	handles	Claim	1 to Many
Policy	is subject of	DiscountRule	1 to Many
Policy + User	logs calculation in	PremiumCalculationLog	Many to 1 each

11. API Documentation

The Spring Boot backend exposes RESTful endpoints documented via Swagger UI at /swagger-ui.html. All endpoints except /api/auth/** require a valid JWT Bearer token. Role restrictions are enforced at the service layer via Spring Security method security.

11.1 Authentication (/api/auth)

Method	Endpoint	Description
POST	/register	Register a new user account. Returns userId and generated JWT token.
POST	/login	Authenticate credentials. Returns JWT token with embedded role claim.

11.2 Users (/api/v1/users)

Method	Endpoint	Description
GET	/{id}	Get full user profile including points, level, badges, and role.
GET	/leaderboard	Get ranked list of all users ordered by userPoints descending.
GET	/role/{role}	Filter and list users by their assigned role.
PUT	/{id}/status	Admin: toggle user account active/inactive status.

11.3 Quizzes and Attempts (/api/v1/quizzes, /api/v1/attempts)

Method	Endpoint	Description
GET	/quizzes	List all available quizzes with category and difficulty.
POST	/quizzes	Admin: create a new quiz with category and difficulty settings.
GET	/questions/quiz/{id}	Get all questions and answers for a specific quiz.
POST	/attempts	Submit quiz answers. Returns score, points earned, and newly unlocked badges.
GET	/education	List all educational articles and video content from the learning center.

11.4 Policies and Premiums

Method	Endpoint	Description
GET	/policies	List all active insurance policy products available for purchase.
POST	/policies	Admin: create a new insurance policy product.
GET	/{userId}/premium/calculate/{policyId}	Calculate personalized premium including full discount breakdown for a user.
POST	/{userId}/policies	Purchase a policy for a user. Creates UserPolicy in PENDING_UNDERWRITING status.
PUT	/{userId}/policies/{userPolicyId}/pay	Complete payment and activate policy (post-underwriter approval).

11.5 Claims (/api/v1/claims)

Method	Endpoint	Description
POST	/file	Customer: file a new insurance claim with incident details and claim type.
GET	/user/{userId}	Get all claims filed by a specific user.
PUT	/{id}/process	Claims Officer: approve or reject a claim with a reviewed amount or reason.

Method	Endpoint	Description
PUT	/{id}/settle	Claims Officer: confirm final settlement amount and close the claim.

11.6 Badges, Rewards and Discount Rules

Method	Endpoint	Description
GET	/badges	List all badges with point thresholds and descriptions.
GET	/badges/user/{userId}	Get all badges earned by a specific user.
GET	/rewards	List all redeemable rewards with point costs and expiry dates.
POST	/rewards/{id}/redeem?userId={id}	Customer: redeem a reward by spending userPoints.
GET	/discount-rules	List all active discount rules and their threshold conditions.
POST	/discount-rules	Admin: create a new discount rule for the premium calculation engine.

11.7 Admin Pipeline (/api/v1/admin/pipeline)

Method	Endpoint	Description
GET	/policies	Master list of all user policy applications across all statuses.
PUT	/policies/{id}/assign	Admin: assign a pending policy application to a specific Underwriter.
PUT	/policies/{id}/quote	Underwriter: submit a final premium quote for a policy application.
GET	/claims	Admin: view all claim submissions for assignment oversight.
PUT	/claims/{id}/assign	Admin: assign a submitted claim to a specific Claims Officer.

12. Frontend Component Architecture

The Angular 21 frontend uses standalone components with lazy loading via app.routes.ts. All backend communication is centralized through ApiService using RxJS Observables. Authentication state is managed by AuthService with JWT token storage in localStorage and automatic injection via auth.interceptor.ts.

12.1 Public Pages

Component	Route	Description
LandingComponent	/	Marketing page introducing IQsure's value proposition with call-to-action for registration.
AboutComponent	/about	Platform overview, mission statement, and team information.
LoginComponent	/login	JWT-based credential entry. On success, decodes JWT role and redirects to the appropriate dashboard.
RegisterComponent	/register	New customer account creation with email uniqueness validation and immediate JWT issue on success.

12.2 Customer Pages

Component	Route	Description
DashboardComponent	/dashboard	Hub displaying points, level progress, active streak, badge count, and quick-action shortcuts.
QuizzesComponent	/quizzes	Grid view of all available quizzes with category filter, difficulty badge, and personal best score.
TakeQuizComponent	/take-quiz/:id	Timed quiz interface with 5-minute countdown, question navigation, and submit-on-timeout.
QuizResultComponent	/quiz-result	Post-quiz summary: score, points earned, speed bonus breakdown, and newly unlocked badges.
AchievementsComponent	/achievements	Full catalogue of 8 achievements showing earned/unearned status and unlock conditions.
LeaderboardComponent	/leaderboard	Ranked user table with podium highlight for top 3. Personal rank highlighted in the list.
RewardsComponent	/rewards	Catalog of redeemable rewards. Point balance displayed. Redeem button triggers point deduction.
PoliciesComponent	/policies	Insurance product marketplace with filtering by type and plan category.
ApplyPolicyComponent	/apply-policy	Policy application form with insured member addition and premium preview display.
MyPoliciesComponent	/my-policies	Personal policy vault with status tracking, premium breakdown, and insured member list.
MyClaimsComponent	/my-claims	Personal claim history with real-time status, amounts, and rejection reasons where applicable.

12.3 Admin Pages

Component	Route	Description
AdminDashboardComponent	/admin/dashboard	Platform-wide statistics: user count, active policies, pending claims, quiz participation.
QuizMgmtComponent	/admin/quiz-mgmt	Full CRUD for quizzes, questions, and answers with difficulty and category management.
PolicyMgmtComponent	/admin/policy-mgmt	Insurance product design interface with all policy parameters and activation toggle.

Component	Route	Description
BadgeMgmtComponent	/admin/badge-mgmt	Badge creation and management with point threshold configuration.
DiscountRulesComponent	/admin/discount-rules	Discount rule engine configuration: thresholds, discount percentages, and policy type applicability.
AdminUsersComponent	/admin/users	Master user list with role display and account status management.
AssignUnderwriterComponent	/admin/assign-uw	Assign pending policy applications to available Underwriters.
AssignClaimsOfficerComponent	/admin/assign-co	Assign submitted claims to available Claims Officers.
CustomersComponent	/admin/customers	Filtered view of all ROLE_USER accounts with points and policy count.

12.4 Shared and Service Layer

Element	Type	Description
NavbarComponent	Shared Component	Role-adaptive navigation bar showing menu items appropriate to the authenticated user's role. Hidden on login and register pages.
ApiService	Angular Service	Centralized HTTP abstraction layer with typed Observable-returning methods for every backend endpoint. Base URL configured to http://localhost:8080.
AuthService	Angular Service	Manages JWT storage in localStorage, exposes getCurrentUser() and getUserRole() helpers, and provides logout logic.
auth.interceptor.ts	HTTP Interceptor	Intercepts every outbound HttpClient request and appends Authorization: Bearer {token} header automatically.
AuthGuard	Route Guard	Redirects unauthenticated requests to /login.
AdminGuard	Route Guard	Restricts /admin/** routes to ROLE_ADMIN users only.
UnderwriterGuard	Route Guard	Restricts underwriter routes to ROLE_UNDERWRITER users.
ClaimsOfficerGuard	Route Guard	Restricts claims officer routes to ROLE_CLAIMS_OFFICER users.

13. Key Features and Technical Highlights

13.1 Key Features

• **Gamification Loop:** Quiz performance directly reduces insurance premiums through an automated discount engine. The points, badges, level, streak, achievement, leaderboard, and rewards system form an interlocking engagement loop.

• **Dynamic Premium Engine:** PremiumCalculationService evaluates all active DiscountRules against a user's profile in real time, stacking multiple discounts up to a 50% cap and returning a fully transparent PremiumBreakdown DTO.

• **Speed-Bonus Scoring:** Quiz scoring includes a time-based multiplier (up to +50%) rewarding knowledge fluency. The 5-minute countdown creates urgency and differentiates genuine understanding from reference-based answering.

• **Role-Enforced Pipeline:** Admin, Underwriter, and Claims Officer roles form a structured processing pipeline. Policy applications flow from PENDING_UNDERWRITING to ACTIVE. Claims flow from SUBMITTED through officer review to SETTLED or REJECTED.

• **Transparent Pricing:** PremiumCalculationLog persists every premium calculation with a snapshot of the user's gamification profile at that moment, enabling full auditability of how each final premium was derived.

• **Configurable Rule Engine:** Discount rules are fully configurable by Admins at runtime, with no code changes required to adjust discount thresholds, percentages, or applicable policy types.

• **Comprehensive Security:** BCrypt password hashing, stateless JWT authentication, CORS policy, SQL injection prevention via JPA parameterized queries, and Angular's built-in XSS sanitization provide a defense-in-depth security posture.

• **H2 Embedded Database:** Using H2 eliminates external database dependencies, making the project immediately runnable on any machine with Java and Node.js installed.

13.2 Development Challenges and Solutions

Challenge	Solution Implemented
Linking Gamification to Finance in Real Time	Designed PremiumCalculationService to snapshot the user's current gamification state (points, badges, best score) at calculation time, evaluate all applicable DiscountRules in a single database query, and stack results while enforcing the 50% cap — all within a single synchronous service call.
JWT Role Extraction on Frontend	AuthService decodes the JWT payload from localStorage without a round-trip call, extracting the role claim for immediate route guard decisions and dynamic navbar rendering.
Preventing Point Gaming via Repeated Attempts	QuizAttemptService checks for an existing attempt record with positive pointsEarned before awarding points. Re-attempts are permitted for practice but yield no additional points.
Complex Claim Lifecycle State Management	ClaimService enforces valid status transitions in code (e.g., ASSIGNED cannot jump to SETTLED), throwing an IllegalStateException for invalid transitions to prevent data corruption.
CORS in Development	WebConfig.java explicitly whitelists http://localhost:4200 as an allowed origin, enabling cross-origin requests from the Angular dev server without browser security errors.
H2 In-Memory Data Loss on Restart	DataSeeder.java (ApplicationRunner) re-seeds all required reference data on every application startup, ensuring the platform is fully operational immediately after any restart.

14. Setup and Installation Guide

IQsure requires Java 17+, Maven, Node.js 18+, and Angular CLI to run locally. No external database configuration is needed as H2 is embedded and auto-configured.

14.1 Backend Setup

Step	Command / Action
1. Navigate to project root	cd HTF_IQsure_Capstone_Project-master
2. Build the project	mvn clean install
3. Start the backend server	mvn spring-boot:run
4. Verify startup	API runs on http://localhost:8080
5. Access Swagger UI	Open http://localhost:8080/swagger-ui.html
6. Access H2 Console	Open http://localhost:8080/h2-console — JDBC URL: jdbc:h2:mem:iqsuredb, User: sa, Password: password123
7. Default Admin Login	email: admin@iqsure.com / password: admin123

14.2 Frontend Setup

Step	Command / Action
1. Navigate to frontend	cd frontend
2. Install dependencies	npm install
3. Start the dev server	ng serve
4. Open in browser	http://localhost:4200
5. Run frontend tests	ng test (uses Vitest runner)

14.3 Running Tests

- Backend unit and integration tests: **mvn test** (Spring Boot Test + JUnit)
- Frontend component and service tests: **ng test** (Vitest 4.x)
- API testing: Swagger UI at /swagger-ui.html provides interactive endpoint testing

15. Future Enhancements

The IQsure architecture is designed for extensibility. The following enhancements are identified for future development phases:

Enhancement	Description	Priority
Production Database Migration	Replace H2 embedded database with PostgreSQL or MySQL for production deployments, requiring only a datasource configuration change with no code modifications due to the JPA abstraction.	High
Real-Time Notifications	Integrate WebSocket support via Spring WebSocket and the Angular WebSocket client to deliver real-time claim status updates and badge award notifications without requiring page refresh.	High
OAuth2 Social Login	Add Google and LinkedIn OAuth2 login options via Spring Security OAuth2 client, reducing registration friction and expanding the platform's accessible audience.	Medium
AI-Powered Quiz Generation	Integrate a large language model API (e.g. OpenAI GPT) to dynamically generate new quiz questions on any insurance topic, eliminating the need for manual content creation.	Medium

Enhancement	Description	Priority
Mobile Application	Develop a companion mobile app using Angular with Capacitor (or Ionic Framework) to provide native iOS and Android experiences with offline quiz capability and push notifications.	Medium
Payment Gateway Integration	Integrate a real payment provider (Razorpay, Stripe) for actual premium payment processing, replacing the current simulated payment flow.	High
Advanced Analytics Dashboard	Build an admin analytics module showing quiz completion rates, most common claim types, discount rule effectiveness, and user engagement trends using chart libraries.	Low

16. Conclusion

IQsure successfully demonstrates the transformative potential of applying gamification principles to the traditionally disengaging domain of insurance. By directly linking consumer learning behaviour — quiz completion, badge achievement, and point accumulation — to tangible financial outcomes in the form of personalized premium discounts, the platform creates a genuine incentive for insurance literacy that no conventional educational approach has achieved.

Built as a capstone project for The Hartford training program, IQsure delivers a complete, production-ready full-stack web application on a Spring Boot 4.x backend with Spring Security, Spring Data JPA, and H2 database, paired with an Angular 21 frontend using RxJS, SCSS, Ionicons, TypeScript interfaces, and a robust JWT authentication pipeline. The system supports four distinct user roles — Customer, Admin, Underwriter, and Claims Officer — each with purpose-built dashboards and enforced access controls.

The platform's configurable discount rule engine, comprehensive claims processing pipeline, full administrative content management capability, and transparent premium calculation audit logging position IQsure as a technically rigorous and commercially viable solution to a real industry problem. The architecture is deliberately designed for extensibility, with a clear pathway to production deployment, real payment integration, and AI-augmented quiz generation in future phases.

— END OF DOCUMENT —